

# State-Space Models: Expanded Study Notes

From Continuous Dynamics to S4, Mamba, GPUs, and Production Systems

Aligned to <https://www.youtube.com/watch?v=A7iAmVr0QE4>

Companion notebook: `SSM_S4_Mamba_HiPPO.ipynb`

Figures auto-generated in `State_Space_Models/figures/`

August 2026 — long-form edition ( $\geq 25$  pages)

**How to use this book-like note.** Read in order. Every equation is surrounded by explanation and, where useful, a derivation. Figures are not decoration: each one is explained in the caption and in the text above/below it. When you finish a part, you should be able to retell it without looking.

## Contents

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| <b>1</b> | <b>The problem this whole field is trying to solve</b>                | <b>3</b>  |
| 1.1      | What sequence models must do . . . . .                                | 3         |
| <b>2</b> | <b>Continuous-time SSMs — full story and derivations</b>              | <b>5</b>  |
| 2.1      | The defining equations . . . . .                                      | 5         |
| 2.2      | Homogeneous solution and the matrix exponential . . . . .             | 6         |
| 2.3      | Forced solution — variation of parameters (full derivation) . . . . . | 7         |
| 2.4      | Output as convolution . . . . .                                       | 8         |
| <b>3</b> | <b>Discretization — from continuous time to tokens</b>                | <b>8</b>  |
| 3.1      | Zero-order hold (exact under a staircase input) . . . . .             | 8         |
| 3.2      | Bilinear / Tustin method . . . . .                                    | 10        |
| 3.3      | What $\Delta$ means in a deep network . . . . .                       | 11        |
| <b>4</b> | <b>Unrolling: recurrence, convolution, FFT</b>                        | <b>11</b> |
| 4.1      | Unrolling the recurrence . . . . .                                    | 11        |
| 4.2      | Diagonal kernels as multi-scale memory . . . . .                      | 11        |
| 4.3      | FFT training for LTI models . . . . .                                 | 12        |
| <b>5</b> | <b>HiPPO — principled memory initialization</b>                       | <b>13</b> |
| <b>6</b> | <b>Structured SSMs (S4 family) — making kernels computable</b>        | <b>14</b> |
| 6.1      | Why structure? . . . . .                                              | 14        |
| 6.2      | Diagonal SSMs (S4D) . . . . .                                         | 15        |

|           |                                                        |           |
|-----------|--------------------------------------------------------|-----------|
| 6.3       | DPLR and Woodbury (classical S4)                       | 15        |
| 6.4       | H3 and Hyena — the missing multiplicative piece        | 15        |
| <b>7</b>  | <b>Mamba — selective state spaces</b>                  | <b>15</b> |
| 7.1       | Where LTI fails                                        | 15        |
| 7.2       | Selection mechanism                                    | 16        |
| 7.3       | Parallel scan algebra                                  | 17        |
| 7.4       | Mamba block architecture                               | 18        |
| 7.5       | Mamba paper findings (what was actually claimed/shown) | 18        |
| 7.6       | Mamba-2 and state space duality (SSD)                  | 19        |
| <b>8</b>  | <b>GPU theory — why “linear FLOPs” is not enough</b>   | <b>19</b> |
| 8.1       | Memory hierarchy                                       | 19        |
| 8.2       | Why Mamba-2 chases tensor cores                        | 20        |
| 8.3       | Numerics that show up in real training                 | 20        |
| <b>9</b>  | <b>Multi-channel layout and stacking into LMs</b>      | <b>20</b> |
| <b>10</b> | <b>Code: educational implementations</b>               | <b>20</b> |
| 10.1      | LTI path — bilinear, ZOH-diag, kernel, scan, FFT       | 20        |
| 10.2      | Selective scan (clarity over speed)                    | 21        |
| 10.3      | Production libraries                                   | 22        |
| <b>11</b> | <b>Industry use cases and real-life implementation</b> | <b>23</b> |
| 11.1      | Where SSMs and hybrids actually appear                 | 23        |
| 11.2      | A realistic serving story (hybrid LM)                  | 23        |
| 11.3      | When to choose what (product checklist)                | 24        |
| 11.4      | Training recipe notes used in practice                 | 24        |
| <b>12</b> | <b>Research timeline recap (findings in one place)</b> | <b>24</b> |
| <b>13</b> | <b>Limits and open problems</b>                        | <b>25</b> |
| <b>14</b> | <b>Study plan and exercises</b>                        | <b>25</b> |
| <b>15</b> | <b>Symbol glossary</b>                                 | <b>26</b> |
| <b>16</b> | <b>References</b>                                      | <b>26</b> |

# 1 The problem this whole field is trying to solve

## 1.1 What sequence models must do

A sequence model sees a stream  $u_0, u_1, \dots, u_{L-1}$  (token embeddings, audio samples, DNA bases, image patches, ...) and must produce outputs  $y_0, \dots, y_{L-1}$  that depend on the *right* parts of the past. Three pressures fight each other:

1. **Quality** — do not forget important information from far away.
2. **Compute** — train on billions of tokens and serve users with low latency.
3. **Memory** — fit activations, weights, and caches on a GPU.

Transformers address quality with pairwise attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V. \quad (1)$$

Each position builds a query vector  $Q$  (“what am I looking for?”), a key  $K$  (“what do I contain?”), and a value  $V$  (“what content do I pass if selected?”). The matrix  $QK^\top$  has shape  $L \times L$ : every token scores every other token. Dividing by  $\sqrt{d}$  keeps the scale of those scores from exploding as the channel dimension  $d$  grows, so softmax does not collapse to a one-hot. The result is a rich, content-based routing of information.

The cost is structural. Forming and multiplying through an  $L \times L$  matrix is  $\Theta(L^2)$  work (and naive memory). At generation time, implementations store past keys and values in a **KV-cache** so they are not recomputed; that cache still grows as  $\Theta(L)$  *per layer*, which becomes the bottleneck for long chats, long documents, and multi-thousand concurrent sessions.

**Intuition.** Attention re-reads the entire room every time someone speaks. That is flexible, but the room gets expensive as more people walk in.

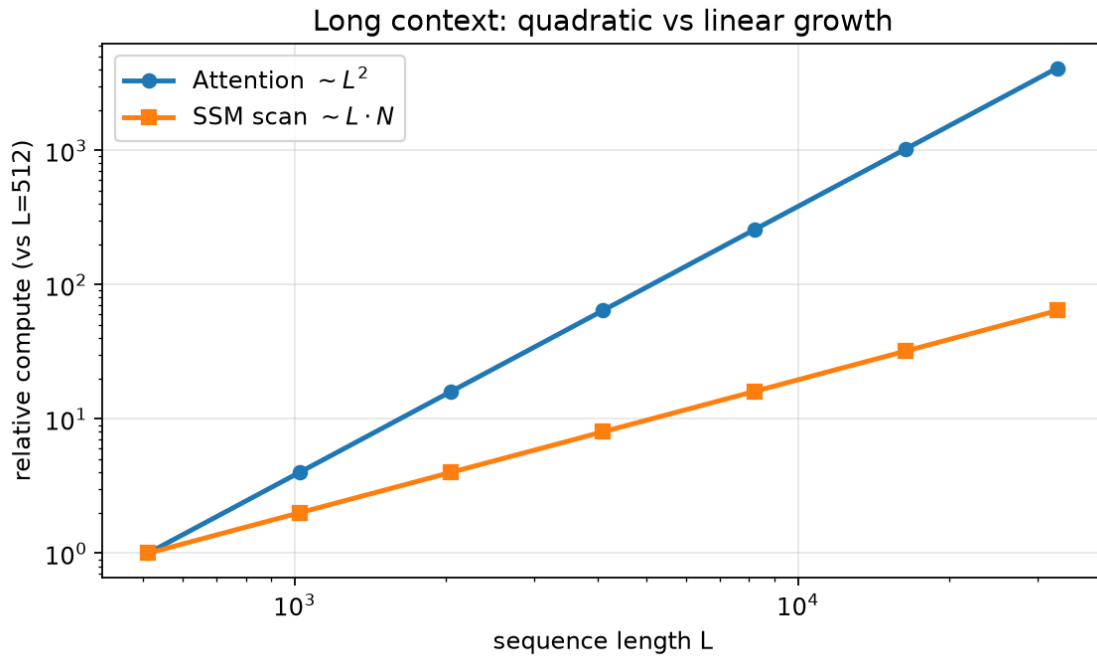


Figure 1: **Figure: compute growth with sequence length.** Both curves are normalized to their value at  $L = 512$  so you can compare *shapes*. Attention’s  $L^2$  curve climbs steeply on the log-log plot: doubling  $L$  multiplies pairwise work by about four. An SSM scan that scales like  $L \cdot N$  (state size  $N$  fixed) climbs much more slowly. This is the core systems motivation for SSMs on long context: not that they are “always better,” but that their cost curve is a different animal.

RNNs and LSTMs already had fixed-size states and  $O(1)$  update per token, but training is sequential along time, gradients vanish or explode over long horizons, and on large language workloads they historically lost to Transformers. State-space models keep the fixed-state idea, import continuous-time memory theory, then add structure and (later) selection so that training can be parallel and quality can approach attention on many tasks.

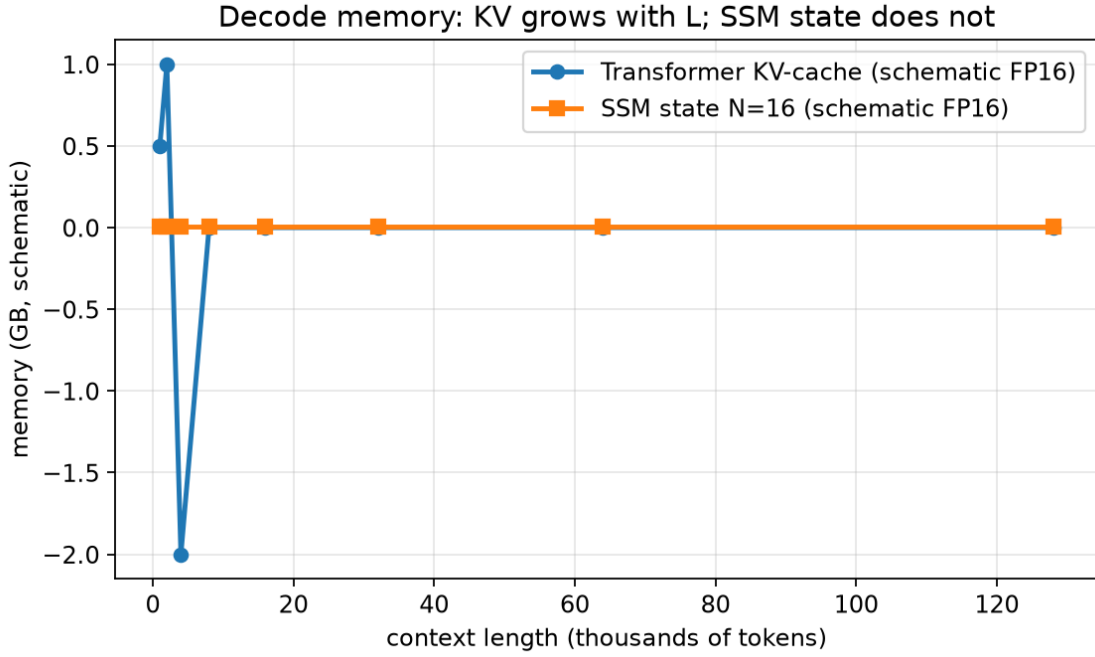


Figure 2: **Figure: decode memory, schematic.** We plot a rough FP16 estimate for a 32-layer, width-4096 model. Transformer KV-cache grows linearly with context length (two matrices  $K, V$  per layer). An SSM recurrent state of size  $N = 16$  per channel is essentially *flat* in  $L$ . Real systems differ by GQA/MQA, quantization, hybrid layers, etc., but the qualitative gap is why Mamba-style models advertise large serving wins on long contexts.

## 2 Continuous-time SSMs — full story and derivations

### 2.1 The defining equations

A continuous linear state-space model is

$$\begin{aligned} \dot{x}(t) &= Ax(t) + Bu(t), \\ y(t) &= Cx(t) + Du(t). \end{aligned} \tag{2}$$

Here:

- $u(t) \in \mathbb{R}$  is the input (we start SISO; multi-channel later).
- $x(t) \in \mathbb{R}^N$  is the **state** — a fixed-size notebook of the past.
- $y(t) \in \mathbb{R}$  is the output feature.
- $A \in \mathbb{R}^{N \times N}$  is the free dynamics of the notebook (how it fades and mixes alone).
- $B \in \mathbb{R}^{N \times 1}$  aims the pen that writes  $u$  into the notebook.
- $C \in \mathbb{R}^{1 \times N}$  reads a linear combination of slots into  $y$ .
- $D \in \mathbb{R}$  is feedthrough: a direct skip from input to output (residual-like).

**Line 1 in words.** The rate of change of memory equals “what memory does by itself” plus “what the new input writes.”

**Line 2 in words.** The output is a readout of memory, optionally plus a peek at the raw input.

## 2.2 Homogeneous solution and the matrix exponential

If  $u \equiv 0$ , then  $\dot{x} = Ax$ . The solution is  $x(t) = e^{At}x(0)$ , where the matrix exponential is defined by the power series

$$e^{At} = \sum_{k=0}^{\infty} \frac{(At)^k}{k!}, \quad (3)$$

which always converges. If  $A$  is diagonalizable,  $A = V\Lambda V^{-1}$  with  $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_N)$ , then

$$e^{At} = V \text{diag}(e^{\lambda_1 t}, \dots, e^{\lambda_N t}) V^{-1}. \quad (4)$$

Each eigenmode multiplies by  $e^{\lambda_i t}$ :

- $\text{Re}(\lambda_i) < 0$ : stable decay (forgetting);
- $\text{Re}(\lambda_i) \approx 0$ : long-lived memory;
- $\text{Im}(\lambda_i) \neq 0$ : oscillation / ringing;
- $\text{Re}(\lambda_i) > 0$ : explosion (usually disastrous inside a network).

For a real pole  $\lambda = -a$  with  $a > 0$ , the continuous time-constant is  $1/a$ : after time  $t = 1/a$  the mode has dropped by factor  $e^{-1}$ .

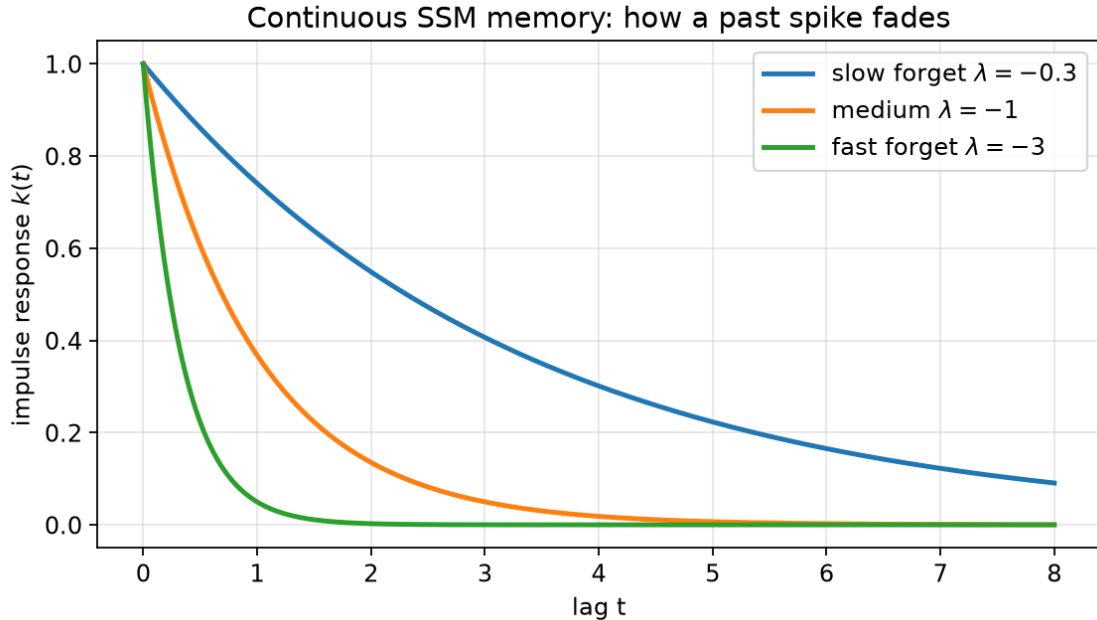


Figure 3: **Figure: real poles and forgetting speed.** For the scalar system  $\dot{x} = \lambda x + u$ ,  $y = x$  with  $\lambda < 0$ , the impulse response is simply  $k(t) = e^{\lambda t}$  for  $t \geq 0$ . More negative  $\lambda$  means the spike is forgotten faster. When you stack many modes (next sections), the full kernel is a mixture of such curves — that is multi-scale memory.

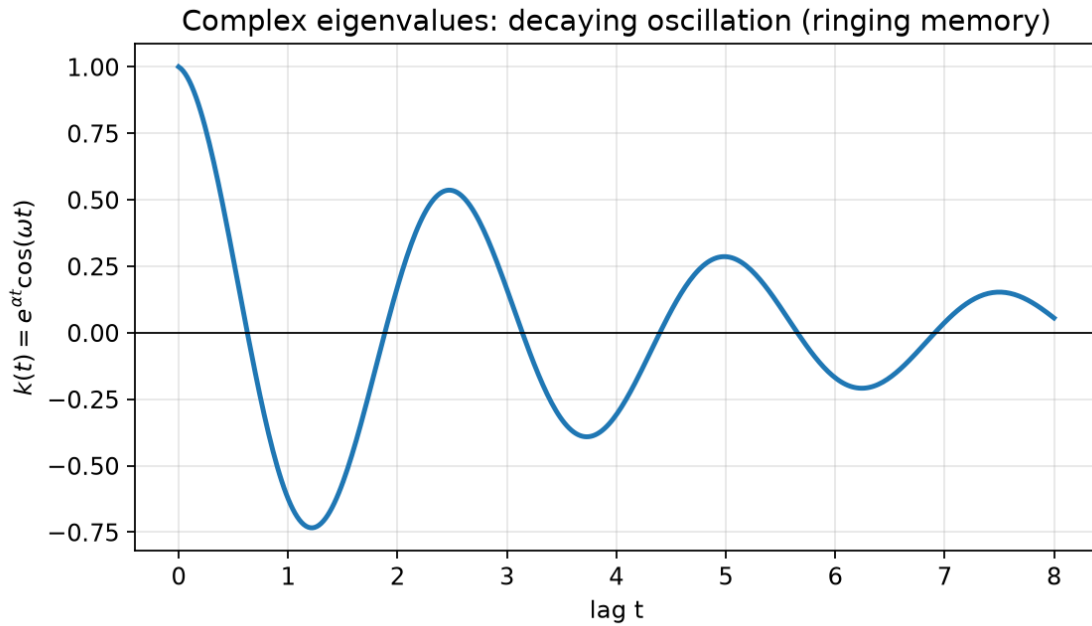


Figure 4: **Figure: complex poles.** If eigenvalues come in complex conjugate pairs  $\alpha \pm i\omega$  with  $\alpha < 0$ , the impulse response rings:  $e^{\alpha t} \cos(\omega t + \phi)$ . This can represent periodic structure in signals (audio, seasons, rhythmic patterns). In language models one often constrains poles to be real-negative for stability simplicity, but the general theory allows oscillation.

### 2.3 Forced solution — variation of parameters (full derivation)

We solve  $\dot{x} = Ax + Bu$  with  $x(0) = x_0$ .

**Step 1.** Seek  $x(t) = e^{At}v(t)$  (homogeneous solution shape with varying coefficient).

**Step 2.** Differentiate:

$$\dot{x} = Ae^{At}v + e^{At}\dot{v}. \quad (5)$$

Set equal to  $Ax + Bu = Ae^{At}v + Bu$ :

$$e^{At}\dot{v} = Bu \quad \Rightarrow \quad \dot{v} = e^{-At}Bu. \quad (6)$$

**Step 3.** Integrate from 0 to  $t$ :

$$v(t) = v(0) + \int_0^t e^{-A\tau}Bu(\tau) d\tau. \quad (7)$$

But  $x(0) = e^0v(0) = v(0) = x_0$ , so

$$x(t) = e^{At}x_0 + \int_0^t e^{A(t-\tau)}Bu(\tau) d\tau. \quad (8)$$

**Words for (8).** Fade the initial notebook for time  $t$ . Then, for every past moment  $\tau$ , take the write  $Bu(\tau)$  and fade it by the elapsed time  $t - \tau$ ; add all those faded writes together.

## 2.4 Output as convolution

Set  $x_0 = 0$  and ignore  $D$  for the pure SSM path:

$$y(t) = \int_0^t C e^{A(t-\tau)} B u(\tau) d\tau = (k * u)(t), \quad (9)$$

with causal impulse response

$$k(t) = \begin{cases} C e^{At} B & t \geq 0, \\ 0 & t < 0. \end{cases} \quad (10)$$

So an LTI SSM is completely characterized by  $k$ : poke the system once at time 0;  $k(t)$  is what you still see at lag  $t$ .

**Transfer function.** Taking Laplace transforms,  $Y(s) = G(s)U(s)$  with

$$G(s) = C(sI - A)^{-1}B + D. \quad (11)$$

Poles of  $G$  are eigenvalues of  $A$ . Designing memory timescales *is* placing those poles.

## 3 Discretization — from continuous time to tokens

Language and most deep-learning pipelines are discrete: indices  $k = 0, 1, 2, \dots$ . We choose a positive stepsize  $\Delta$  meaning “how much continuous time one token is worth,” and we want

$$x_k = \bar{A}x_{k-1} + \bar{B}u_k, \quad y_k = \bar{C}x_k + \bar{D}u_k. \quad (12)$$

### 3.1 Zero-order hold (exact under a staircase input)

Assume  $u(t)$  is constant on each interval  $[k\Delta, (k+1)\Delta)$  and equal to  $u_k$ . Integrate (8) over one step with constant  $u$ :

$$x((k+1)\Delta) = e^{A\Delta}x(k\Delta) + \left( \int_0^\Delta e^{A\tau} d\tau \right) B u_k. \quad (13)$$

Hence

$$\bar{A} = e^{A\Delta}, \quad \bar{B} = \left( \int_0^\Delta e^{A\tau} d\tau \right) B. \quad (14)$$

If  $A$  is invertible,

$$\int_0^\Delta e^{A\tau} d\tau = A^{-1}(e^{A\Delta} - I), \quad (15)$$

so  $\bar{B} = A^{-1}(e^{A\Delta} - I)B$ .

**Scalar worked example.** Let  $N = 1$ ,  $A = -a$  ( $a > 0$ ),  $B = 1$ ,  $C = 1$ ,  $D = 0$ . Then

$$\bar{A} = e^{-a\Delta}, \quad \bar{B} = \frac{1 - e^{-a\Delta}}{a}, \quad K_m = \bar{C}\bar{A}^m\bar{B} = \frac{1 - e^{-a\Delta}}{a} e^{-a\Delta m}. \quad (16)$$



The discrete half-life (steps until a spike’s effect halves) is

$$m_{1/2} = \frac{\ln 2}{a\Delta}. \quad (17)$$

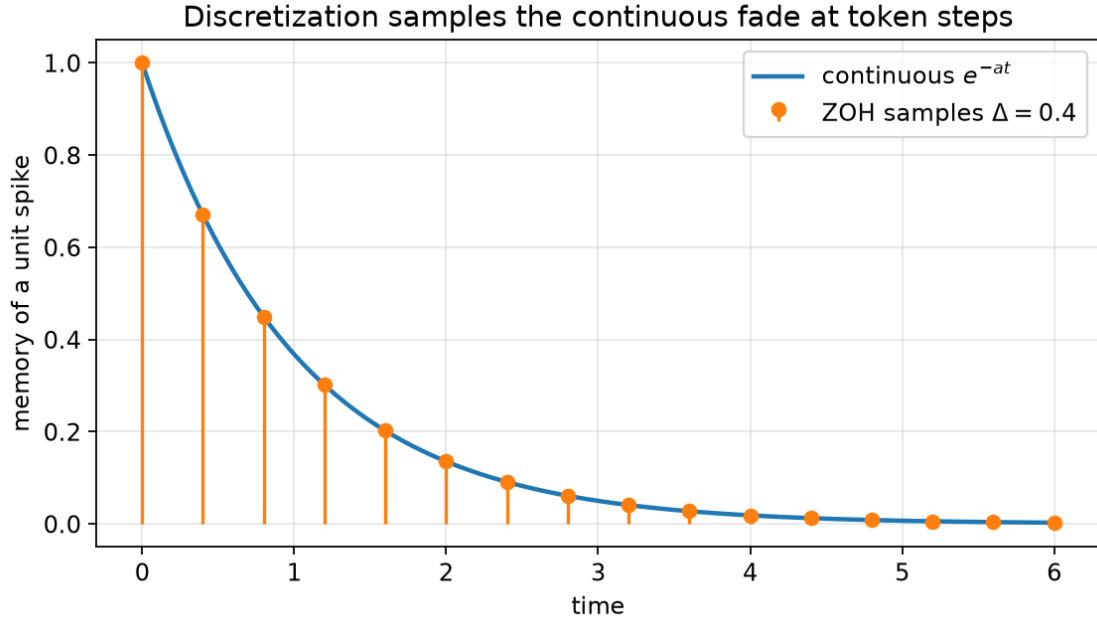


Figure 5: **Figure: continuous fade vs discrete samples.** The smooth curve is continuous exponential memory. Stem markers are what a ZOH discretization “sees” at token times  $0, \Delta, 2\Delta, \dots$ . Discretization does not invent a new theory of memory; it samples the continuous theory at the rate tokens arrive.

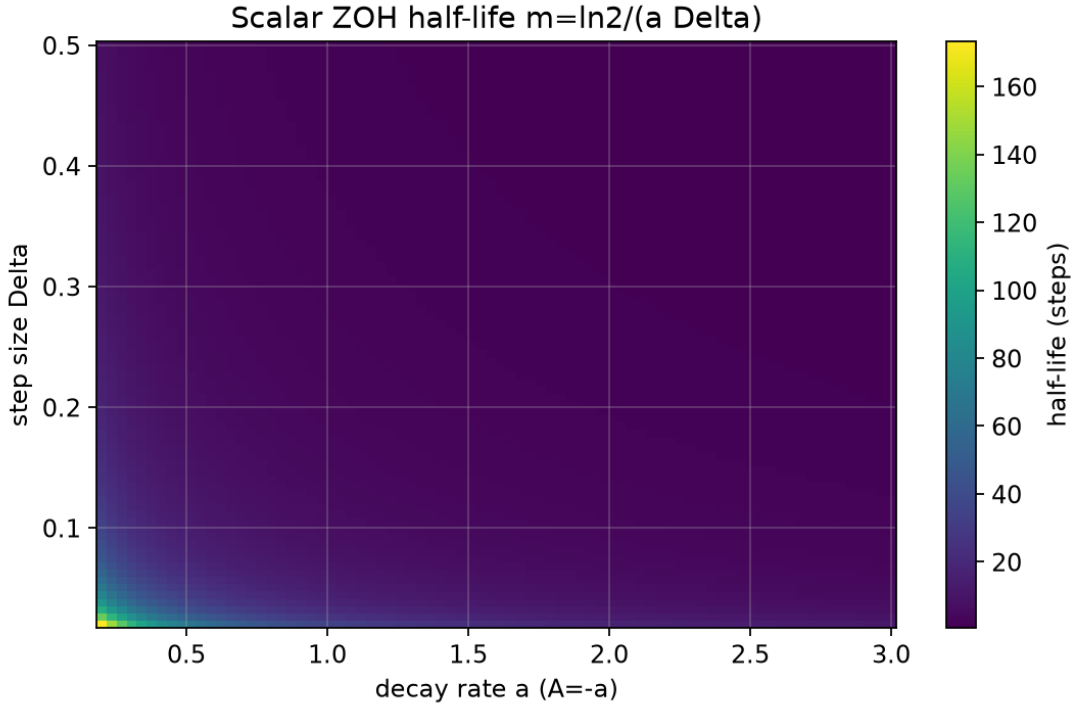


Figure 6: **Figure: half-life heatmap.** Color shows  $m_{1/2} = \ln 2 / (a\Delta)$  for the scalar leaky integrator. Moving right (larger decay rate  $a$ ) or up (larger step  $\Delta$ ) shortens memory in token units. Mamba’s selection can be read as moving around this heatmap *per token*: important tokens may use a different effective  $\Delta$  than noise tokens.

**Diagonal ZOH (implementation form).** If  $A = \text{diag}(\lambda_i)$ , componentwise

$$\bar{A}_{ii} = e^{\lambda_i \Delta}, \quad \bar{B}_i = \begin{cases} \frac{e^{\lambda_i \Delta} - 1}{\lambda_i} B_i & \lambda_i \neq 0, \\ \Delta B_i & \lambda_i = 0. \end{cases} \quad (18)$$

In floating point use `expm1` for stability near zero.

### 3.2 Bilinear / Tustin method

The bilinear transform maps continuous Laplace  $s$  to discrete  $z$  by

$$s = \frac{2}{\Delta} \frac{1 - z^{-1}}{1 + z^{-1}}. \quad (19)$$

Applying this idea to the state equation yields the common form used in S4 literature:

$$\bar{A} = \left(I - \frac{\Delta}{2} A\right)^{-1} \left(I + \frac{\Delta}{2} A\right), \quad \bar{B} = \left(I - \frac{\Delta}{2} A\right)^{-1} \Delta B. \quad (20)$$

**Trapezoid derivation sketch.** Approximate

$$\frac{x_k - x_{k-1}}{\Delta} \approx A \frac{x_k + x_{k-1}}{2} + B \tilde{u}_k \quad (21)$$

and rearrange for  $x_k$  in terms of  $x_{k-1}$ . You solve a linear system of size  $N$  instead of computing a matrix exponential series. Stability of continuous poles is mapped in a controlled way under the bilinear map (no aliasing of the entire left half-plane into unstable discrete poles the way some crude maps can).

### 3.3 What $\Delta$ means in a deep network

Large  $\Delta$ : each token is a “big moment”; the discrete map moves state harder; old content is easier to overwrite. Small  $\Delta$ : gentle updates; the notebook is protected. Classical S4 treats  $\Delta$  as a parameter (possibly learned) but **not** a function of the current token. Mamba’s central modeling idea is  $\Delta_k = \Delta(u_k)$ .

## 4 Unrolling: recurrence, convolution, FFT

### 4.1 Unrolling the recurrence

Assume  $x_{-1} = 0$ . Iterate (12):

$$x_0 = \bar{B}u_0, \quad (22)$$

$$x_1 = \bar{A}\bar{B}u_0 + \bar{B}u_1, \quad (23)$$

$$x_k = \sum_{j=0}^k \bar{A}^{k-j} \bar{B}u_j. \quad (24)$$

Reading with  $\bar{C}$  (and omitting  $D$  for a moment),

$$y_k = \sum_{j=0}^k K_{k-j} u_j, \quad K_m = \bar{C} \bar{A}^m \bar{B}. \quad (25)$$

So the length- $L$  kernel is the list

$$K = (\bar{C}\bar{B}, \bar{C}\bar{A}\bar{B}, \dots, \bar{C}\bar{A}^{L-1}\bar{B}), \quad (26)$$

and  $y = K * u$  (causal convolution).

**Two faces of one LTI model. Recurrent face:** update  $x$  each step — natural for autoregressive decode ( $O(1)$  state). **Convolutional face:** build  $K$  once and filter the whole sequence — natural for parallel training.

### 4.2 Diagonal kernels as multi-scale memory

If  $\bar{A} = \text{diag}(\bar{\lambda}_i)$ , then

$$K_m = \sum_{i=1}^N C_i \bar{\lambda}_i^m \bar{B}_i. \quad (27)$$

Each term is a geometric mode: a pure exponential decay (or oscillation if complex) on the lag axis. Summing modes with different rates builds a kernel that remembers some things briefly and some

things for a long time.

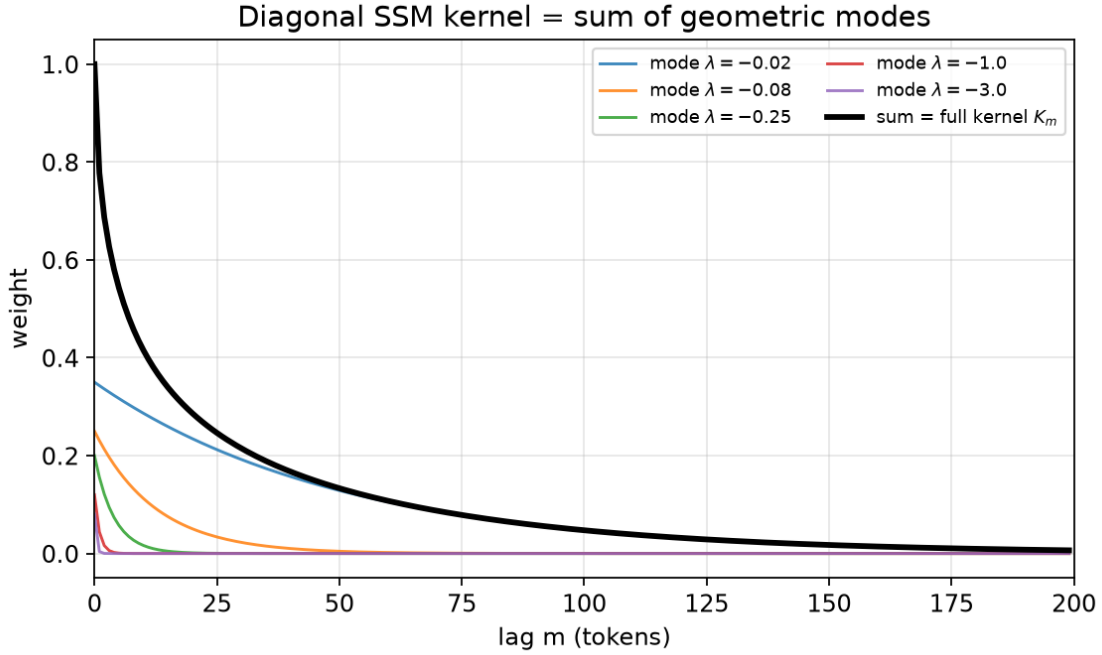


Figure 7: **Figure: sum of geometric modes.** Thin curves are individual modes (fast and slow). The thick black curve is their weighted sum — the full kernel  $K_m$ . This is the mental picture for S4D/diagonal SSMs: memory is a bank of faders, not a mysterious black box.

### 4.3 FFT training for LTI models

Naive convolution is  $O(L^2)$ . The convolution theorem says that after discrete Fourier transform,

$$K * u = \mathcal{F}^{-1}(\mathcal{F}(K) \odot \mathcal{F}(u)), \quad (28)$$

provided we zero-pad to length at least  $2L - 1$  (or the next power of two) so that circular convolution agrees with linear convolution on the first  $L$  outputs. Cost becomes  $O(L \log L)$ .

**Generating-function view (bridge to S4 papers).**

$$\hat{K}(z) = \sum_{m=0}^{L-1} K_m z^m = \bar{C}(I - z\bar{A})^{-1}\bar{B} \quad (29)$$

(up to endpoint conventions). Evaluating  $\hat{K}$  at roots of unity and inverse-FFT-ing is another way to apply the filter. When  $\bar{A}$  is structured (diagonal or DPLR),  $(I - z\bar{A})^{-1}$  becomes cheap, leading to Cauchy-kernel algorithms in classical S4.

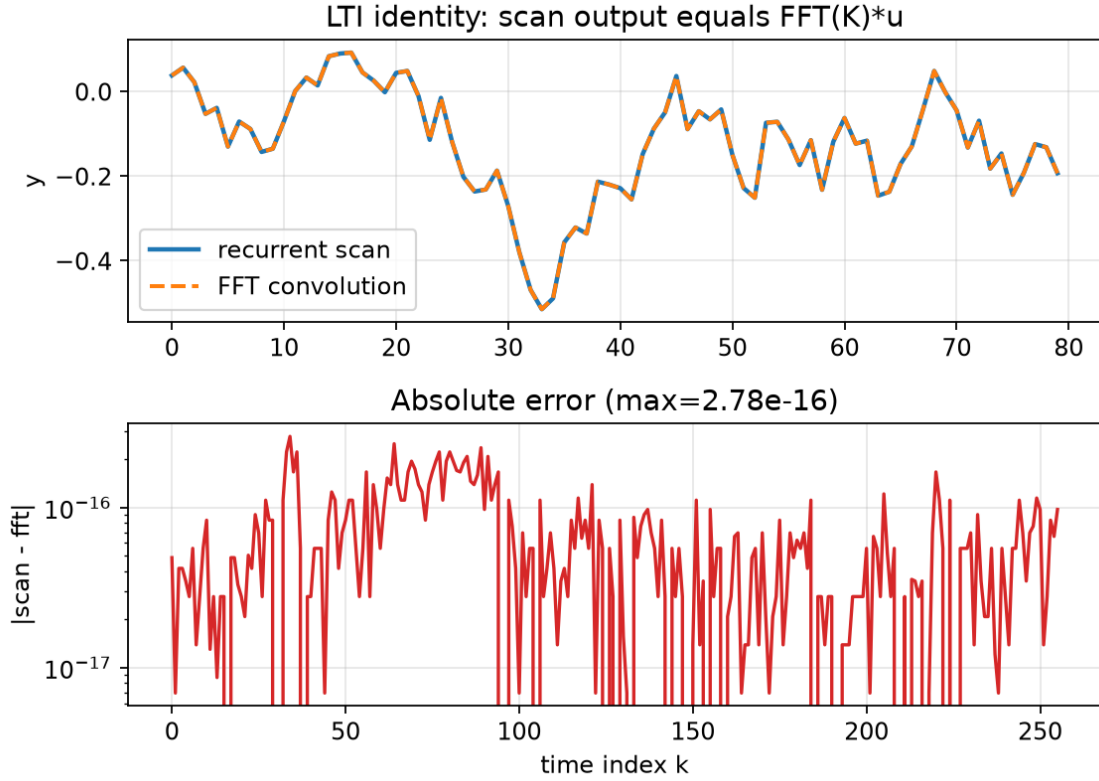


Figure 8: **Figure: the LTI identity in numbers.** Top: outputs of a recurrent scan and of FFT-based convolution with the analytic kernel, for a random diagonal SSM and random input (first 80 steps shown). Bottom: absolute difference on a log scale — only floating-point noise. If your educational code does not reproduce this near-zero error, a bug exists in discretization, kernel construction, or padding. This identity is the mathematical heart of classical SSM training.

**Hard boundary.** FFT training requires a *single fixed* kernel  $K$  for the whole sequence. If  $\bar{A}_k$  and  $\bar{B}_k$  change with  $k$  (Mamba selection), there is no global  $K$ , and this section’s training path does not apply.

## 5 HiPPO — principled memory initialization

Random  $A$  matrices usually implement either short memory or unstable dynamics. **HiPPO** (High-order Polynomial Projection Operators; Gu et al., NeurIPS 2020) asks a precise question: what linear ODE makes the state  $x(t)$  track the coefficients of an optimal online projection of the *history* of  $u$  onto a polynomial basis?

The answer is a specific family of matrices  $A$  (and input vectors  $B$ ). Low-order coefficients store coarse long-term trends; higher-order coefficients store fine recent detail. That is multi-scale compression with a theorem behind it, not only a heuristic.

One common construction (HiPPO-LegT; sign conventions vary slightly by paper/code) is

$$A_{nk} = \begin{cases} -\sqrt{(2n+1)(2k+1)} & n > k, \\ -(n+1) & n = k, \\ 0 & n < k. \end{cases} \quad (30)$$

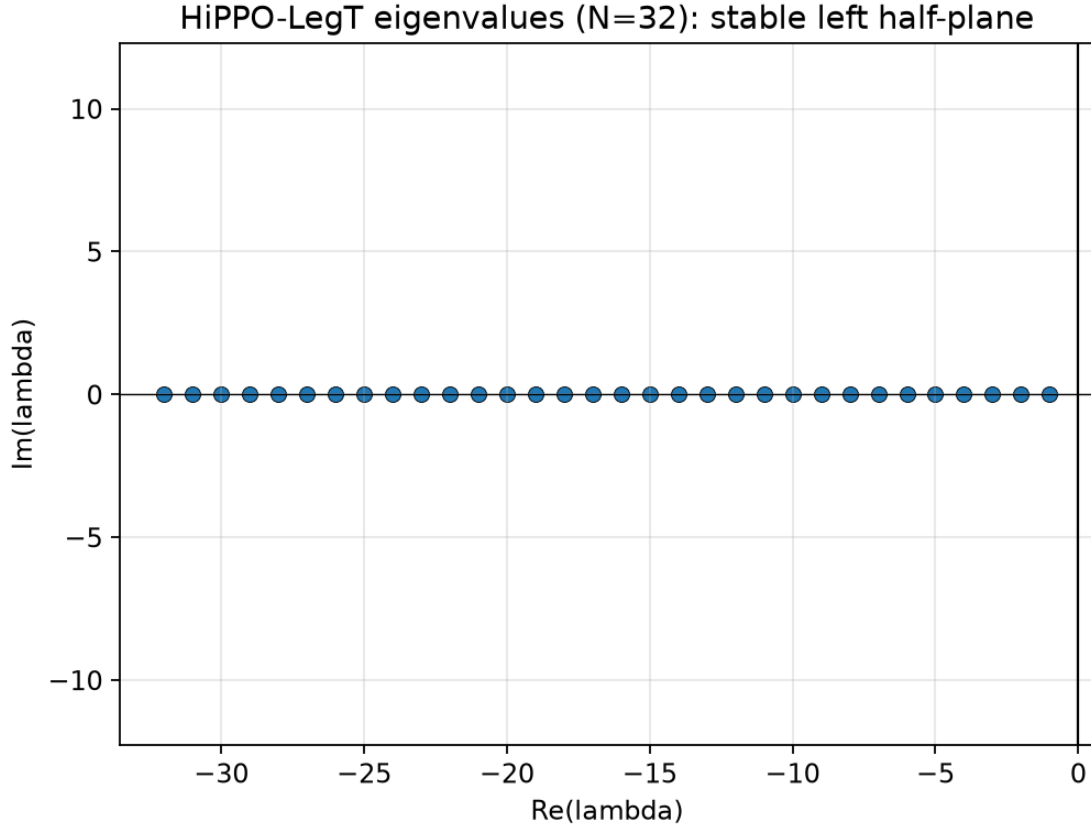


Figure 9: **Figure: HiPPO spectrum.** Eigenvalues of a  $32 \times 32$  HiPPO-LegT matrix, plotted in the complex plane. All real parts are negative (left half-plane): the continuous system is stable. The spread of real parts is exactly the multi-scale lifetime structure: some modes die fast, some slowly. When you initialize an SSM from HiPPO, you are planting this spectrum (or a structured cousin of it) into the network before learning begins.

**Paper-level finding.** HiPPO showed large gains on long memorization tasks relative to naive recurrent baselines, establishing that *memory quality is largely a structure/initialization problem on  $A$ .*

## 6 Structured SSMs (S4 family) — making kernels computable

### 6.1 Why structure?

Even a beautiful HiPPO  $A$  is dense. Forming powers  $\bar{A}^m$  or evaluating generating functions costs too much and can be numerically unstable for large  $N$  and  $L$ . S4 (Gu, Goel, Ré, ICLR 2022) forces a structure that keeps expressivity while making  $\hat{K}(z)$  evaluable quickly.

## 6.2 Diagonal SSMs (S4D)

Set  $A = \text{diag}(\lambda_i)$  (often complex) with careful initialization (HiPPO eigenvalues or S4D-Lin/Inv recipes). Then  $K_m = \sum_i C_i \bar{\lambda}_i^m \bar{B}_i$  as above. **Finding:** diagonal models capture most practical S4 gains with vastly simpler code. This is what you should implement first, and it is the substrate under many selective models.

## 6.3 DPLR and Woodbury (classical S4)

HiPPO-class matrices can be written, after unitary transform, as diagonal plus low rank:

$$A = \Lambda - PQ^*. \quad (31)$$

The Woodbury identity says that if you know  $M^{-1}$  and you add a low-rank bump  $UV^*$ ,

$$(M + UV^*)^{-1} = M^{-1} - M^{-1}U(I + V^*M^{-1}U)^{-1}V^*M^{-1}. \quad (32)$$

Applied to resolvents  $(zI - A)^{-1}$ , DPLR systems reduce to diagonal resolvents plus a thin correction. Those diagonal pieces produce **Cauchy kernels** of the form  $1/(\omega_j - \zeta_i)$ , for which stable fast algorithms exist. That pipeline is why S4 could claim near-linear kernel computation with HiPPO-level memory theory.

**S4 layer as a training story.** For each channel: parameters  $(A, B, C, \Delta, D) \rightarrow$  discretize  $\rightarrow$  build or evaluate  $K \rightarrow y = \text{FFTConv}(K, u) + Du \rightarrow$  nonlinear channel mixing  $\rightarrow$  stack depth. At inference: drop the FFT and run the recurrence with stored state.

**S4 paper finding.** Strong Long Range Arena results at sequence lengths where contemporary Transformers struggled, putting SSMs on the deep learning map.

## 6.4 H3 and Hyena — the missing multiplicative piece

H3 (“Hungry Hungry Hippos”) and Hyena-style architectures showed that pure linear filters are not enough for language: you also need **data-controlled multiplicative interactions** (gating) stacked with long convolutions. This foreshadows Mamba, which moves content dependence into the SSM parameters themselves rather than only into external gates (Mamba still uses gates too).

# 7 Mamba — selective state spaces

## 7.1 Where LTI fails

Consider a stream of noise tokens, a rare marker, then values you must copy later. A fixed kernel  $K$  applies the same lag weights regardless of content: it cannot open a register only when the marker appears. Transformers can content-address; pure S4 cannot as flexibly. Language is full of such content-dependent memory demands.

## 7.2 Selection mechanism

Mamba (Gu & Dao, 2023) sets

$$\begin{aligned}\Delta_k &= \text{softplus}(W_\Delta u_k + b), \\ B_k &= W_B u_k, \\ C_k &= W_C u_k,\end{aligned}\tag{33}$$

then discretizes *per step* to  $\bar{A}_k, \bar{B}_k$  and runs the time-varying recurrence

$$x_k = \bar{A}_k x_{k-1} + \bar{B}_k u_k, \quad y_k = C_k x_k.\tag{34}$$

$A$  itself usually remains constrained for stability, e.g. a negative diagonal

$$A = -\exp(A_{\log}) \quad (\text{elementwise}).\tag{35}$$

### Reading selection as policy.

- Large  $\Delta_k$ : aggressive update;  $\bar{A}_k$  can shrink toward 0 along modes  $\Rightarrow$  forget / reset.
- Small  $\Delta_k$ :  $\bar{A}_k \approx I + \Delta_k A \approx I \Rightarrow$  hold state.
- $B_k$ : which state directions to write.
- $C_k$ : which directions to read now.

For diagonal ZOH with  $\lambda_i < 0$ ,

$$\bar{A}_{k,ii} = e^{\lambda_i \Delta_k} \in (0, 1).\tag{36}$$

As  $\Delta_k \rightarrow \infty$ ,  $\bar{A}_{k,ii} \rightarrow 0$  (wipe). As  $\Delta_k \rightarrow 0$ ,  $\bar{A}_{k,ii} \rightarrow 1$  (preserve). That is selection as linear algebra.



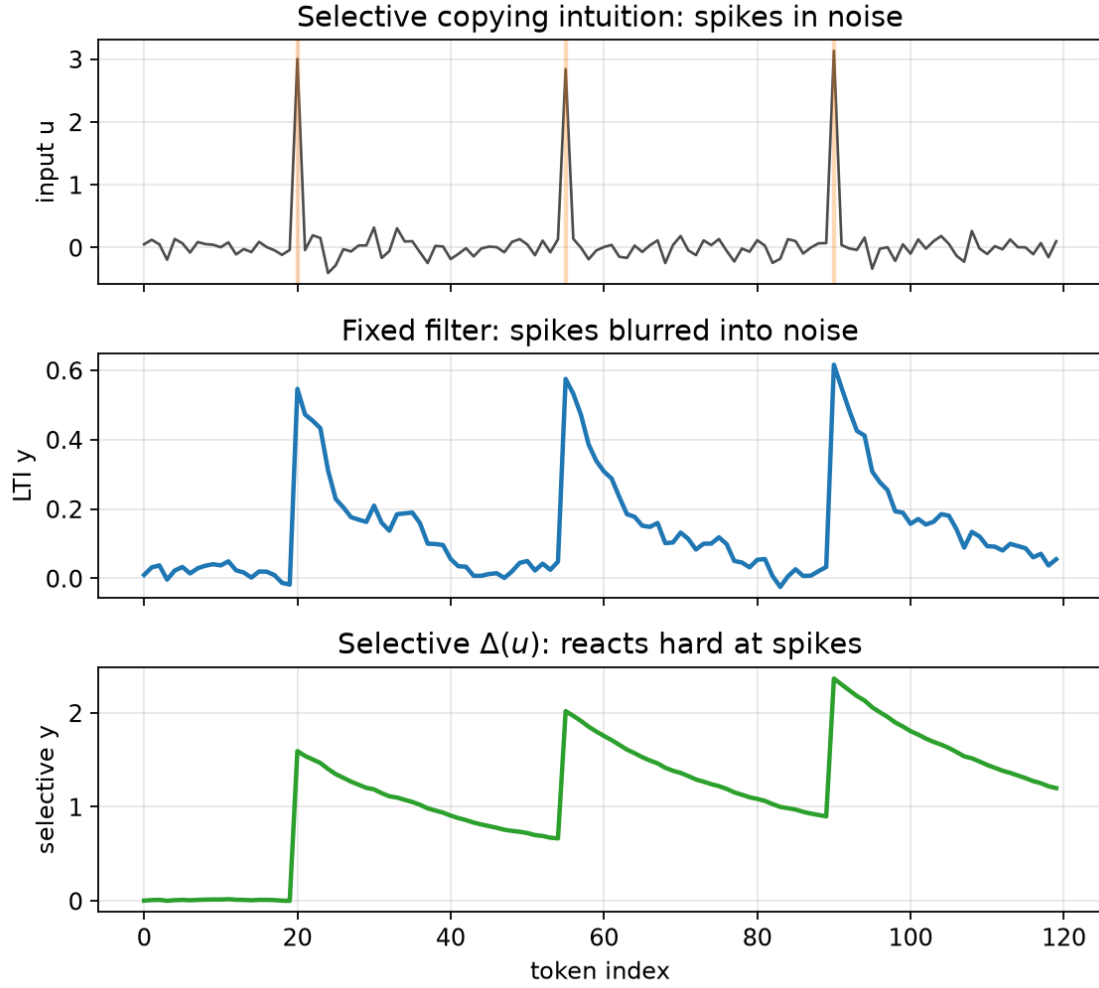


Figure 10: **Figure: LTI vs selective on spikes-in-noise (toy).** Top: input with three large spikes in Gaussian noise. Middle: fixed- $\Delta$  LTI diagonal SSM — spikes are smoothed; noise continually pollutes the same filter. Bottom:  $\Delta$  enlarged when  $|u|$  is large — the output reacts sharply at spikes and stays calmer on noise. This is a cartoon of why selection helps “ignore fillers, keep events,” not a full language benchmark.

### 7.3 Parallel scan algebra

Identify step  $k$  with the affine map  $x \mapsto \bar{A}_k x + b_k$ ,  $b_k = \bar{B}_k u_k$ . Composition of two maps (diagonal case elementwise) is

$$(\bar{A}_2, b_2) \otimes (\bar{A}_1, b_1) = (\bar{A}_2 \odot \bar{A}_1, \bar{A}_2 \odot b_1 + b_2). \quad (37)$$

This binary operator is associative with identity  $(1, 0)$ . Therefore the Blelloch parallel prefix algorithm computes all prefix maps in  $O(\log L)$  parallel depth and  $O(L)$  work — exact, not approximate.

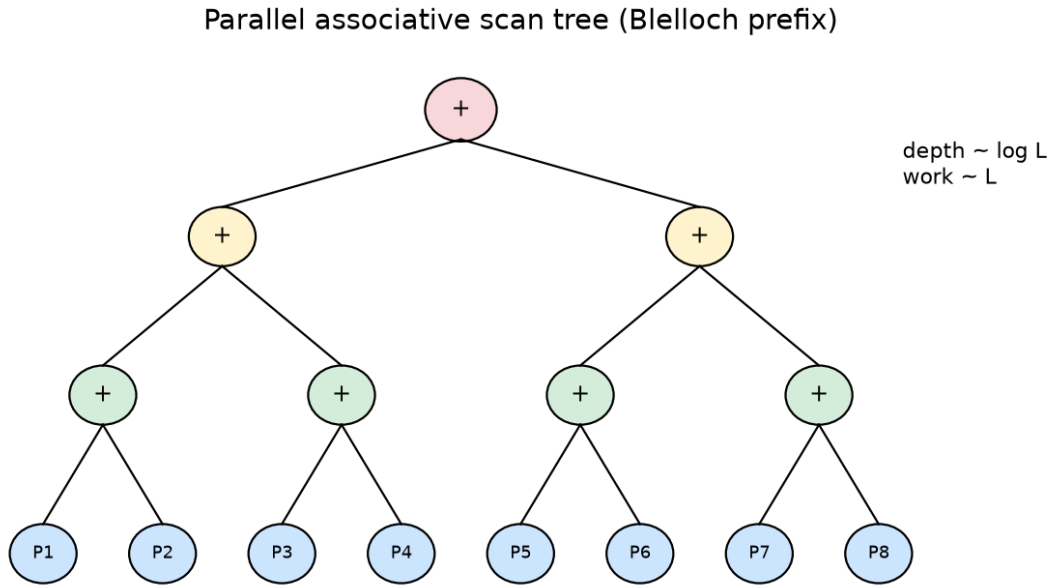


Figure 11: **Figure: scan as a tree.** Leaves  $P_1, \dots, P_8$  are per-step maps. Internal nodes combine children with  $\otimes$  from (37). Because  $\otimes$  is associative, any parenthesization is valid, so the tree can run in parallel across the sequence. Depth scales like  $\log L$ ; total combine work scales like  $L$ .

## 7.4 Mamba block architecture

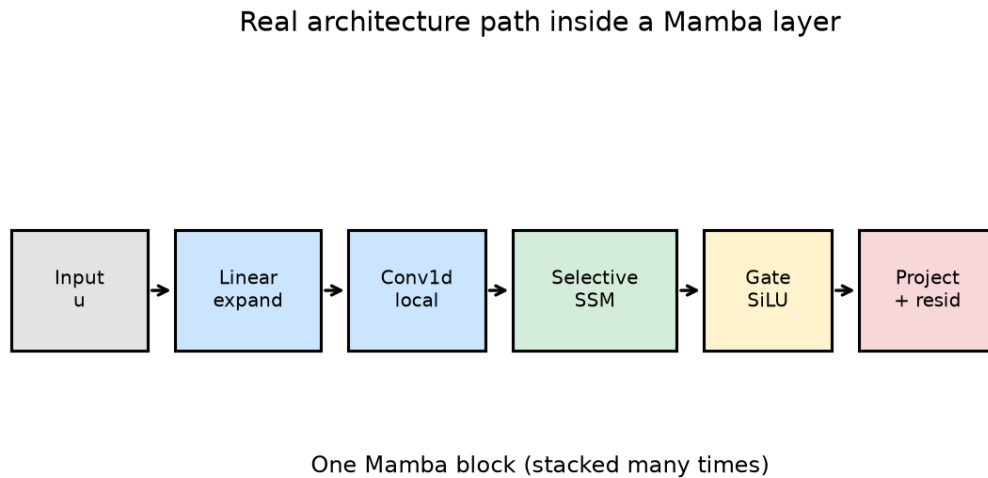


Figure 12: **Figure: one Mamba block.** Left to right: expand channels; local depthwise Conv1d (short syntax); selective SSM (long content-aware memory); SiLU gate branch; project back and add residual. A language model stacks many such blocks. There is no growing KV-cache: only a fixed recurrent state plus a tiny conv state.

## 7.5 Mamba paper findings (what was actually claimed/shown)

- Language modeling on The Pile competitive with Transformer baselines in the 130M–2.8B range.
- Strong behavior as sequence length grows; large inference throughput from  $O(1)$  state updates.

- Synthetic tasks (selective copy, induction-style) where LTI SSMs fail.
- Beyond text: audio and genomics experiments demonstrating modality generality.

## 7.6 Mamba-2 and state space duality (SSD)

Dao & Gu (2024) show selective SSMs dualize to structured (semiseparable) matrix multiplies related to linear attention. Practically: chunk the sequence, do heavy work as batched GEMMs (tensor cores), pass a light state across chunks. Results: larger practical  $N$  (e.g. 64–128 class), faster training than pure Mamba-1 scans, cleaner multi-GPU projection layouts. Minimal SSD educational cores shrink to tens of lines of PyTorch; production still uses highly optimized kernels.

# 8 GPU theory — why “linear FLOPs” is not enough

## 8.1 Memory hierarchy

GPU memory hierarchy (why fused Mamba scan exists)

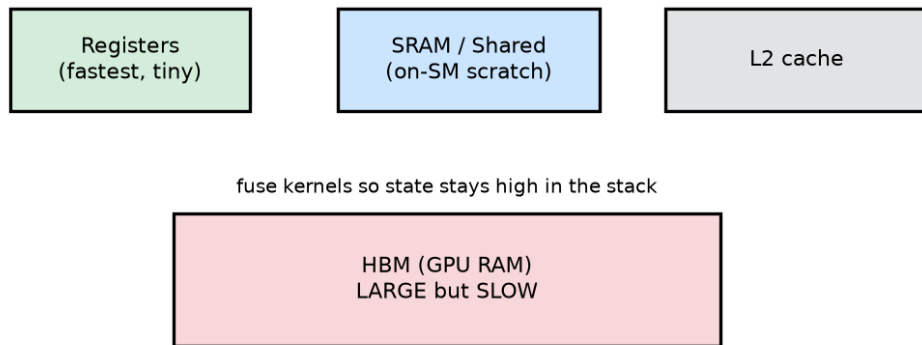


Figure 13: **Figure: GPU memory stack.** Registers and SRAM (shared memory) are tiny but extremely fast. HBM (the big GPU RAM) is huge but comparatively slow and bandwidth-limited. If every step of a scan reads/writes full activations and states through HBM, you become bandwidth-bound and wall-clock collapses even when big- $O$  FLOPs look linear. Fused Mamba kernels keep the hot state in SRAM, stream sequence tiles, and recompute intermediates on the backward pass (FlashAttention philosophy).

**Roofline reminder.** Arithmetic intensity = FLOPs/bytes moved from HBM. Low intensity  $\Rightarrow$  bandwidth-bound (common). High intensity  $\Rightarrow$  compute-bound (large tensor-core GEMMs). Naive PyTorch loops over  $L$  launch tiny ops and thrash HBM: educational, not production.

**IO sketch.** For batch  $B$ , length  $L$ , width  $D$ , state  $N$ , a fused implementation aims for HBM traffic on the order of reading inputs and writing outputs  $O(BLD)$  (plus modest factors), **not**

materializing full state histories  $O(BLDN)$  every layer.

## 8.2 Why Mamba-2 chases tensor cores

Tensor cores accelerate dense matrix multiplies far more than ad-hoc scan loops. SSD chunking turns much of the selective work into GEMMs of shape involving chunk length  $C$  (often 256) and state size  $N$ . That is why larger  $N$  becomes attractive: it becomes a matmul dimension, not only a slower sequential burden.

## 8.3 Numerics that show up in real training

- Parameterize  $A$  to stay stable (negative real parts).
- Softplus + clamp/floor on  $\Delta$ .
- Prefer FP32 master weights even if compute is BF16/FP16.
- Expect tiny differences between parallel scan and sequential reference under mixed precision; test with tolerances.
- Complex diagonal forms need careful real/imag packing.

## 9 Multi-channel layout and stacking into LMs

Token embeddings are vectors  $u_k \in \mathbb{R}^d$ . Standard design: run SISO SSMs **depthwise** on each channel (or on expanded channels), each with state size  $N$ , then mix channels with linear layers and nonlinearities — the same pattern as depthwise separable convolution.

A pure Mamba LM looks roughly like:

1. token embedding (+ optional positionless design; recurrence carries order);
2. stack of  $L_{\text{ayers}}$  residual Mamba blocks;
3. RMSNorm / LayerNorm variants as in modern LMs;
4. untied or tied LM head to vocabulary logits.

Typical published sizes (original Mamba checkpoints) follow GPT-3-like width/depth tables from 130M to 2.8B trained on The Pile, plus larger community and industry hybrids later.

## 10 Code: educational implementations

### 10.1 LTI path — bilinear, ZOH-diag, kernel, scan, FFT

```
import numpy as np

def discretize_bilinear(A, B, C, D, delta):
    I = np.eye(A.shape[0])
    B1 = np.linalg.inv(I - 0.5 * delta * A)
    Ab = B1 @ (I + 0.5 * delta * A)
    Bb = B1 @ (delta * B)
    return Ab, Bb, C, D
```

```

def discretize_zoh_diag(lam, B, delta):
    """lam, B: shape (N,)"""
    Abar = np.exp(lam * delta)
    Bbar = np.where(
        np.abs(lam) < 1e-8,
        B * delta,
        B * (np.expm1(lam * delta) / lam),
    )
    return Abar, Bbar

def ssm_kernel_diag(Abar, Bbar, C, D, L):
    K = np.zeros(L)
    v = Bbar.copy()
    for m in range(L):
        K[m] = C @ v
        v = Abar * v
    K[0] += D
    return K

def ssm_scan_diag(Abar, Bbar, C, D, u):
    x = np.zeros_like(Bbar)
    ys = []
    for uk in u:
        x = Abar * x + Bbar * uk
        ys.append(C @ x + D * uk)
    return np.asarray(ys)

def fft_conv_causal(K, u):
    L = len(u)
    n = 1
    while n < 2 * L:
        n *= 2
    y = np.fft.irfft(np.fft.rfft(K, n) * np.fft.rfft(u, n), n)
    return y[:L]

# Sanity: y_scan and fft_conv_causal(K,u) must match for LTI systems

```

## 10.2 Selective scan (clarity over speed)

```

import torch

def selective_scan_edu(A_log, B, C, delta, u):
    """
    A_log: (N,) -> lam = -exp(A_log)
    B,C: (L,N); delta:(L,); u:(L,)
    """
    lam = -torch.exp(A_log)
    x = torch.zeros_like(lam)
    ys = []
    for k in range(u.shape[0]):
        Abar = torch.exp(lam * delta[k])
        Bbar = torch.where(

```

```

        lam.abs() < 1e-6,
        B[k] * delta[k],
        B[k] * torch.exp1(lam * delta[k]) / lam,
    ) * u[k]
    x = Abar * x + Bbar
    ys.append((C[k] * x).sum())
return torch.stack(ys)

```

This loop is for learning. Real training/serving uses fused CUDA/Triton kernels from `mamba-ssm` / vendor stacks. If you benchmark the loop against FlashAttention Transformers, you are not measuring Mamba — you are measuring Python.

### 10.3 Production libraries

```

# Official block with fused kernels (CUDA)
from mamba_ssm import Mamba, Mamba2
block = Mamba(d_model=768, d_state=16, d_conv=4, expand=2).cuda()
y = block(x)  # x: (batch, length, d_model)

# Hugging Face causal LM
from transformers import AutoModelForCausalLM, AutoTokenizer
name = "state-spaces/mamba-130m-hf"
tok = AutoTokenizer.from_pretrained(name)
lm = AutoModelForCausalLM.from_pretrained(name)
ids = tok("State space models are", return_tensors="pt")
out = lm.generate(**ids, max_new_tokens=64)
print(tok.decode(out[0], skip_special_tokens=True))

```

**Meaning of hyperparameters.** `d_model`: channel width  $d$ . `d_state`: state size  $N$  (notebook dimension). `d_conv`: local conv kernel width. `expand`: inner expansion factor  $e$  (width becomes  $e \cdot d$  inside the block).

**Fine-tuning note.** LoRA module names differ from Transformers: often `in_proj`, `x_proj`, `out_proj`, embeddings — not `q_proj`/`v_proj`.

## 11 Industry use cases and real-life implementation

Industry path: long context in, efficient SSM/hybrid out

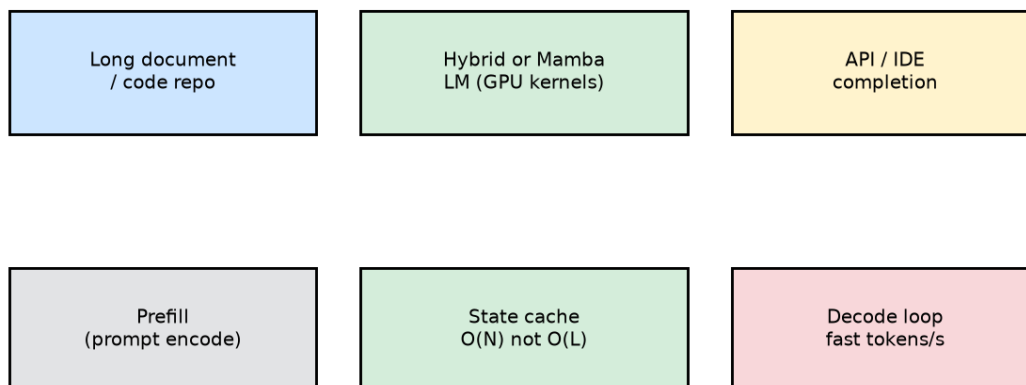


Figure 14: **Figure: deployment sketch.** Long documents or repositories enter a Mamba or hybrid model. Prefill encodes the prompt into states (and any attention caches for hybrid layers). Decode steps update a compact state and emit tokens to an API or IDE. The economic win is concurrent long sessions without multi-gigabyte KV growth per user.

### 11.1 Where SSMs and hybrids actually appear

| System / line          | Org           | How SSMs are used                                                                |
|------------------------|---------------|----------------------------------------------------------------------------------|
| state-spaces/mambaopen | open search   | Pure Mamba LMs for baselines, long-context experiments, distillation starts      |
| Codestral Mamba        | Mistral       | Code generation with Mamba-2 lineage; long repo context; fast completion         |
| Jamba / Jamba 1.5      | AI21          | Hybrid Mamba+Transformer (+MoE): enterprise long documents on leaner GPU budgets |
| Granite hybrid lines   | IBM           | Hybrid efficiency for regulated enterprise serving (watsonx-class stacks)        |
| Nemotron-H-class       | NVIDIA        | High-throughput hybrid inference recipes                                         |
| Vim / VMamba / U-Mamba | research+apps | Vision and medical imaging with spatial scans                                    |
| Audio / genomics SSMs  | research+bio  | Long waveforms and base-pair sequences                                           |

### 11.2 A realistic serving story (hybrid LM)

1. **Model choice.** If the product needs strong ICL and exact quote-from-context, pick a hybrid (most layers SSM, some attention), not pure Mamba.

2. **Kernels.** Install GPU kernels matching your stack (**mamba-ssm**, TensorRT-LLM, vLLM support matrix, vendor NIM). Without them, latency goals fail.
3. **Prefill.** Encode the prompt. SSM layers advance state; attention layers build KV only for those layers (much smaller than full-Transformer cache if attention is sparse in depth).
4. **Decode.** For each new token: update SSM states  $O(N)$  per channel group; update attention KV only for attention layers; sample next token.
5. **Batching.** Continuous batching still applies; memory headroom from flat SSM state often allows higher concurrency.
6. **Quantization.** Weight-only 8/4-bit common; validate quality — selective dynamics can be numerically sensitive.
7. **Observability.** Track tokens/s, GB/s, TTFT, TBT, and quality probes (needle-in-haystack, copy, ICL suite).

### 11.3 When to choose what (product checklist)

**Lean SSM / hybrid** when p95 context is long, concurrency is high, decode VRAM dominates cost, or the modality is a long signal (audio, DNA, logs).

**Transformer-first** when short turns dominate, peak few-shot tool-format quality is the KPI, or your entire MLOps toolchain assumes Llama-class models only.

### 11.4 Training recipe notes used in practice

- Initialize  $A$  stable (HiPPO eigenvalues / negative diagonal recipes from papers).
- Bound  $\Delta$ ; use softplus.
- Keep FP32 parameters when possible.
- Use fused kernels from day one for any serious run.
- Gradient checkpointing for long  $L$ .
- Copy published width/depth/state tables before inventing new ones.
- For hybrids, tune SSM:Attn ratio on *your* retrieval/ICL probes, not only on perplexity.

## 12 Research timeline recap (findings in one place)

1. **HiPPO (2020)** — polynomial projection theory for  $A$ ; memory is a structure problem.
2. **LSSL (2021)** — linear state-space layers unify recurrent/conv/continuous views; compute still hard.
3. **S4 (2022)** — DPLR + Woodbury + Cauchy + FFT; LRA breakthrough.
4. **S4D/DSS (2022)** — diagonal is enough for most gains.
5. **H3/Hyena (2023 era)** — need multiplicative content interactions for language.



6. **Mamba (2023)** — selection + fused scan; competitive LMs; modality breadth.
7. **Mamba-2/SSD (2024)** — duality with structured attention; tensor-core algorithms; larger  $N$ .
8. **Hybrids (2024–)** — production Pareto frontier for general LLMs often mixes SSM + attention.

## 13 Limits and open problems

- Associative recall / exact far copy still favor attention.
- Some ICL and rigid format-following behaviors are stronger in Transformers or hybrids.
- Best kernels remain most mature on NVIDIA CUDA; portability improving via compilers/SSD structure.
- Quantization and extreme selection can interact badly with stability.
- Ecosystem (serving knobs, speculative decoding maturity, adapter zoos) still denser for Transformers.

Correct slogan: SSMs rewrite the **cost curve of long state**, often mixed with attention.  
 Incorrect slogan: SSMs already replace Transformers everywhere.

## 14 Study plan and exercises

### Ten-day path.

1. Explain (2) and  $k(t)$  out loud.
2. Derive (8) without notes.
3. Scalar ZOH half-life; reproduce Figure 13 intuition.
4. Implement diagonal LTI; prove scan  $\approx$  FFT (Figure 5).
5. Plot multi-scale kernels (Figure 4).
6. Read HiPPO abstract + inspect eigenvalues (Figure 11).
7. Read S4 abstract; explain DPLR in one paragraph.
8. Educational selective scan; reproduce Figure 6 spirit.
9. Explain GPU hierarchy and why fusion exists (Figure 9).
10. Run HF Mamba generate; write a one-page ship/no-ship memo for your use case.

### Exercises.

1. Derive  $x(t)$  for  $A = -1$ ,  $B = 1$ ,  $u \equiv 1$ ,  $x(0) = 0$ . Interpret each term.
2. Show unrolling yields  $K_m = \bar{C} \bar{A}^m \bar{B}$ .
3. Prove associativity of  $\otimes$  in (37) for diagonal maps.

4. Build a selective-copy dataset; compare LTI vs selective accuracy.
5. Estimate KV GB vs SSM state GB for your favorite open model shape at  $L = 8k, 128k$ .
6. Profile (even roughly) a pure PyTorch selective loop vs a fused kernel if you have CUDA.

## 15 Symbol glossary

|                    |                                                             |
|--------------------|-------------------------------------------------------------|
| $L$                | Sequence length.                                            |
| $N$                | State size (notebook dimension).                            |
| $d / D$            | Model channel width.                                        |
| $A, B, C, D$       | Continuous SSM parameters.                                  |
| $\Delta$           | Discretization step / timescale (input-dependent in Mamba). |
| $\bar{A}, \bar{B}$ | Discrete transition and input maps.                         |
| $K_m$              | Impulse response at lag $m$ (LTI).                          |
| $x_k$              | State after processing token $k$ .                          |
| <b>HBM</b>         | High-bandwidth GPU main memory.                             |
| <b>SRAM</b>        | Fast on-chip scratchpad / shared memory.                    |
| <b>DPLR</b>        | Diagonal Plus Low-Rank structure.                           |
| <b>SSD</b>         | State Space Duality (Mamba-2).                              |
| <b>KV-cache</b>    | Cached Transformer keys/values at decode.                   |

## 16 References

1. Gu et al., *HiPPO*, NeurIPS 2020. [arXiv:2008.07669](https://arxiv.org/abs/2008.07669)
2. Gu et al., *LSSL*, NeurIPS 2021. [arXiv:2110.13985](https://arxiv.org/abs/2110.13985)
3. Gu, Goel, Ré, *S4*, ICLR 2022. [arXiv:2111.00396](https://arxiv.org/abs/2111.00396)
4. Gu et al., diagonal / S4D follow-ups, 2022
5. Fu et al., H3; Poli et al., Hyena, 2023 era
6. Gu & Dao, *Mamba*, 2023. [arXiv:2312.00752](https://arxiv.org/abs/2312.00752)
7. Dao & Gu, *Mamba-2 / SSD*, 2024. [arXiv:2405.21060](https://arxiv.org/abs/2405.21060)
8. AI21 Jamba hybrid reports; IBM Granite hybrid announcements; Mistral Codestral Mamba
9. Vision Mamba / VMamba / medical U-Mamba literature, 2024–
10. Annotated S4: <https://srush.github.io/annotated-s4/>
11. Mamba code: <https://github.com/state-spaces/mamba>

12. HF Mamba: [https://huggingface.co/docs/transformers/model\\_doc/mamba](https://huggingface.co/docs/transformers/model_doc/mamba)
13. Lecture: <https://www.youtube.com/watch?v=A7iAmVr0QE4>

**Closing picture.** A Transformer re-reads the whole room. An SSM carries a pocket notebook whose fade laws began as continuous-time control theory, became computable through S4 structure, became content-aware through Mamba selection, and became fast on GPUs only when engineers respected HBM vs SRAM. Industry often keeps a few attention layers for the moments when a notebook must become a lookup table again.

If you can explain each figure in this document in two sentences, you understand the field well enough to read the papers without drowning.